

LLM agents: how a stateless text-predictor becomes something that decides and acts

How to read this file: PART 1 is GENERAL LLM/agent knowledge (transferable to any system/job/interview). PART 2 (marked "WORKED EXAMPLE") is how the WCX SRE Agent instantiates it. PART 3 is the interview-ready takeaways. Learn the general model; use the repo to make it concrete. **Trust status:** the real agent loop runs INSIDE the Azure SRE Agent managed service, across the REST boundary — its `while` loop is NOT in this repo (you've reasoned about it + seen its evidence in the thread, not the source). `stop_reason` / `tool_use` / `end_turn` are the standard Claude/OpenAI tool-use contract. The 3-prompt-layer + loop-timing details WERE verified against MCP code on Day 3. Trust the mechanics; remember the loop itself is a managed-service internal, not local code. General concepts are stable industry knowledge, current as of 2026 (Claude 5 family, Opus 4.8, Haiku 4.5 era). **Why this file exists:** "how does the agent decide what to do and when to stop" is the conceptual spine everything else hangs on — the single hardest thing to see because it's behind a service boundary. **Related:** `[[kb-streaming-pipeline]]` (how the loop's output reaches the browser), `[[kb-storage-model]]` (where the growing conversation lives), `[[kb-rag-retrieval]]` (one tool the loop calls), `[[kb-distributed-systems-patterns]]` (statelessness, idempotency, retries), `[[kb-claude-api]]` (the concrete Messages-API surface). `[[kb-harness-engineering]]` (the capstone — the loop is L1, tools are L2 of the stack).

PART 0 — WHY THIS MATTERS (philosophy & real-world connection)

- **The deep principle, one line:** intelligence that can only produce text becomes an actor only when wrapped in a loop that executes its decisions — the capability lives in the harness, not the model. Everything below is a consequence of that sentence. The model is a brain in a jar; the loop, the tools, and the transcript are the body.
- **Why this design exists (why not just a smarter model?):** a bigger, smarter next-token predictor is still *structurally* incapable of acting, remembering across calls, or knowing anything past its training cutoff — those aren't quality gaps you fix with more parameters, they're category limits of a stateless pure function. You don't fix "can't restart the service" or "doesn't know today's incident" by training harder; you fix them with architecture: a loop to carry decisions out, tools to reach the world, external memory to persist and to fetch fresh facts. The agent *is* that architecture. That's why the interesting engineering is in the harness, not the weights.
- **Where you meet this in the wild:** you are, right now, being run by exactly such a loop. Claude Code, Cursor, Copilot's agent mode, customer-support bots, autonomous ops/SRE agents (like WCX's), coding agents, research agents — all the same skeleton: stateless model + `while` loop + tools + a growing transcript. By 2026 "agentic" is the default way LLMs ship into products; "call the model once and print the string" is the exception, not the rule. If you build with LLMs, you are building harnesses — so the harness is the part worth understanding deeply, because it's the part that transfers across every product and every provider.
- **What breaks if you don't internalize it:**
 - Treat the model as if it remembers → context bugs: you forget *you* must re-send the history, and the model "forgets" things you never actually sent it.
 - Expect it to "just do" the thing → nothing happens: it only ever emits text; if no code runs on that text, the action never occurs.
 - Ignore the loop's stop condition → infinite loops (you never check for `end_turn`) or premature stops (you halt the moment text appears, cutting off a turn that was still mid tool-call).
 - Assume the role wall is enforced → prompt injection lands: the system-over-user priority is *learned*, not a security boundary, so hostile text in a tool result or user message can impersonate authority.
- **The transferable mental model: LLM decides / code does** — a *stateless oracle* wrapped by a *stateful orchestrator*. Carry this and any agent system (any framework, any vendor, this repo or the next job) decomposes the same way: find the oracle, find the loop, find where state lives.
- **Why it stays true as models improve:** better models make the *decisions* better — sharper tool choices, fewer hallucinations, longer coherent plans. They do not make a pure function stateful or give text the power to execute itself. A smarter oracle is still an oracle. So the loop/tools/memory architecture isn't scaffolding we'll outgrow; it's the permanent shape of turning a predictor into an agent.

PART 1 — GENERAL KNOWLEDGE

1.1 What an LLM actually is (the foundation everything rests on)

An LLM (Large Language Model) is a **next-token predictor**: text in → a probability distribution over the next token → sample one → append it → repeat. That is the whole primitive. Two consequences you must internalize because everything else follows from them:

- It is a **pure function**: same input → same output distribution. It has no memory, no side effects, no clock, no ability to *do* anything in the world. It cannot call an API, read a file, or remember your last message. It only maps text to a probability distribution.
- It is **stateless between calls**. There is no hidden session on the model side. If the model appears to "remember" the conversation, that is because the calling code re-sends the entire history every time (see §1.9).

Everything an agent can *do* comes from the code wrapped around this predictor — not from the model itself. Carry this one line: **the LLM decides; it cannot act**. Acting is always the surrounding code's job.

1.2 Tokens, embeddings, attention, temperature (the four terms you'll keep hitting)

- **Tokens (not words)**: the model reads and writes in word-pieces drawn from a fixed vocabulary (~100k–200k entries depending on model). "unbelievable" might be 3 tokens; a UUID or a chunk of minified code tokenizes *inefficiently* (many tokens per character). **Billing and context limits are counted in tokens, not words or characters** — this is why code and IDs are "expensive." Never estimate token counts with a word-based ruler; each model has its own tokenizer (a given text can be ~1×–1.35× as many tokens across model generations), so count with the provider's token-counting endpoint when it matters. See [[kb-claude-api]].
- **Embeddings (meaning = geometry)**: each token (and each span of text) is mapped to a vector of hundreds-to-thousands of numbers. *Meaning becomes geometry* — semantically similar text lands near each other in that space (the classic `king - man + woman ≈ queen`). This is the exact same mechanism a retrieval tool like `search_relevant_docs` uses to find "relevant" text: embed the query, find nearby vectors. See [[kb-rag-retrieval]].
- **Attention**: at every layer, every token "attends to" every other token, weighted by learned relevance. This is the source of the model's power (it can relate distant parts of the prompt) AND its central cost limit: attention is $O(n^2)$ in sequence length — doubling the context roughly quadruples the compute. This is the physical reason context windows are finite (§1.9).
- **Temperature**: a knob on the sampling step. `0` = always take the highest-probability token (near-deterministic, good for extraction/classification); higher = more random/creative. (Note: some 2026-era frontier models remove `temperature` entirely and expect you to steer with the prompt instead — don't assume it's always available.)

1.3 Why agents exist — the four gaps

The raw LLM has four hard limitations. An **agent** is the software that bridges them:

1. **Frozen knowledge** — training has a cutoff date; it doesn't know today's incident.
2. **No live data** — it can't query your database, logs, or metrics on its own.
3. **Can't take actions** — it can't restart a service, open a ticket, or send a message.
4. **Hallucination** — asked for a fact it lacks, it will confidently make one up.

The one-line definition to memorize: **an agent = LLM decides, agent code does**. The model chooses *what* should happen; the surrounding code makes it *actually* happen and feeds the result back. That division is the whole game.

1.4 The tool-use protocol — a 4-step structured conversation

"Tool use" (a.k.a. "function calling" — that's the general industry term) is how the model reaches the outside world. It is NOT the model executing anything — it's a strict handshake:

1. **Describe the tools** as JSON schemas in the request context: each tool has a `name`, a natural-language `description`, and an `input_schema` (a JSON Schema of its parameters). This description is *prompt engineering* — the model picks tools based on it (vague description → wrong tool call).
2. **The model emits a `tool_use` block** — it names one (or several) tools and fills in the arguments as JSON — **then STOPS. It never runs the tool**. This is the critical mental model: the model's "action" is just *emitting structured text that requests an action*.

3. **The agent/harness/runtime runs the real code** for that tool and captures the result. (The model is idle during this — see §1.6.)
4. **The result is appended as a `tool_result` message and the WHOLE conversation is re-sent** to the model for the next step.

This request→reason→ `tool_use` →execute→ `tool_result` →re-send cycle is the **ReAct pattern** ("Reason + Act" — reason → act → observe, looped) — the model interleaves reasoning with tool calls. It's the conceptual ancestor of essentially every agent framework.

MCP (Model Context Protocol) is worth knowing as a term: originally an Anthropic proposal, it is now an **industry-standard, cross-vendor protocol** (2025→2026) for exposing tools/prompts/resources to models — think of it as "USB-C for tools." An MCP server is a **registry + transport**: it maps a tool name to real code and speaks a standard wire format so any MCP-aware host can call it. A key subtlety: in many MCP setups, **the function signature + docstring + typed field descriptions ARE the schema the model sees** ("signature-as-prompt") — so a sloppy docstring literally becomes a sloppy prompt and causes wrong tool calls.

1.5 The agent LOOP — the central mechanism

The **agent code** runs a real `while` loop. Each iteration it re-sends the full conversation, gets a response, and decides what to do based on **the model's OUTPUT TYPE, not its content**. Providers expose this as a `stop_reason` (Claude) / `finish_reason` (OpenAI) field on the response:

- `stop_reason: tool_use` → the model wants a tool → **run it, append the `tool_result`, loop again.**
- `stop_reason: end_turn` (a.k.a. `stop`) → the model is done → **stop.** (Plus a max-iterations safety cap so a misbehaving model can't loop forever.)

Multi-tool investigations are just the SAME `tool_use` signal fired repeatedly. "The model isn't satisfied and wants another tool" is NOT a special state — it's simply `tool_use` again instead of `end_turn`. The loop rule stays trivially simple: `tool_use` → run + loop; `end_turn` → stop.

CRITICAL — "text output" ≠ "end_turn" (corrected: text-appears→done → stop_reason==end_turn→done): `end_turn` is the *stop_reason* (an output-type flag); text is *content*. A single response can contain **text AND a `tool_use` block together** — narration before a tool call ("Let me check the logs..." + a `get_logs` call). In that case `stop_reason == tool_use` and **the loop CONTINUES**. This is exactly why in a tool-using agent (e.g. Claude Code) you see: text → tool call → more text → another tool call → final answer. Those interleaved text blocks are *narration-with-`tool_use`* iterations; only the FINAL text block (with no accompanying tool call) carries `end_turn`. So the rule is: **a turn ends when `stop_reason == end_turn`, never merely because text appeared.**

1.6 Turns vs LLM-calls, and where the reasoning actually happens

- **A "turn" is NOT one LLM call (corrected: one-question→one-call → one-question→many-calls).** One user question (= one turn) expands into MANY LLM calls: reason → tool → reason → tool → ... → answer. Every call except the last ends in `tool_use`; only the last ends in `end_turn`. This is the **LLM-decides / agent-does division** playing out over time.
- **"Reason, then call a tool, then reason again" is NOT happening inside the LLM — it's a ping-pong across the API boundary.** Per iteration: **the LLM** reads the whole context, reasons, emits `tool_use`, and STOPS. **The agent code** then runs the tool (the LLM is completely idle during this), appends the result, and re-sends the full context. **The LLM** — as a *fresh call with no memory of the previous one* — reads the now-longer context and reasons again from scratch. The model holds NOTHING between calls; every "reasoning step" is a brand-new invocation re-reading the entire growing transcript.

The clean division to carry into any system:

- **LLM = a stateless oracle that DECIDES** (surfaced via `stop_reason`).
- **Agent code = a stateful orchestrator that ACTS, REMEMBERS (via the stored transcript/store), and LOOPS.**
- The agent's *control logic* is nearly memoryless (it just checks `stop_reason` each iteration); the *state* lives in the accumulating transcript, not in the loop variables. See [[kb-storage-model]].

1.7 The framework/landscape map (know the names conceptually, 2026-current)

You don't need to have used these, but you should be able to place them:

- **ReAct** — the reasoning pattern (§1.4), not a library. Underlies everything.
- **Provider-native tool use / function calling** — every major API (Anthropic Claude, OpenAI, Google Gemini) exposes the §1.4 handshake directly. Often you don't need a framework at all — a `while` loop over the Messages API is a complete agent. See

[[kb-claude-api]]).

- **MCP** — the cross-vendor tool/data standard (§1.4). By 2026 this is the default integration surface, not a niche.
- **Orchestration frameworks** — libraries that pre-build the loop, memory, and tool plumbing so you don't hand-roll it:
 - **LangChain / LangGraph** — the most common; LangGraph models the agent as a state graph.
 - **LlamaIndex** — retrieval/RAG-centric agents.
 - **AutoGen** — Microsoft's multi-agent conversation framework.
 - **OpenAI Agents SDK, Anthropic tool use / Claude Agent SDK / managed agents** — vendor-blessed harnesses; the managed variants run the loop *and* host the sandbox for you.
- **Two axes to classify any agent system:**
 - **Single-agent vs multi-agent** — one loop, or a coordinator (orchestrator) that delegates to sub-agents via **handoffs** (each sub-agent has its own context/thread; they share results by message-passing, not shared memory).
 - **Autonomous vs interactive** — fires on a trigger and runs to completion unattended (a webhook/cron job), vs. a human in the loop steering turn-by-turn (a chat UI). The same agent definition can run in both modes; only the *kickoff* and *when it stops for input* differ.

1.8 Prompts: the message-role system and the 3-layer model

"System prompt" is a **ROLE (authority + persistence)**, not a **specificity level**. Keep two axes separate:

1. **Role** — every message carries a role, and the model was *trained* to weight them differently:
 - **system** — standing rules / persona / safety; highest authority; persists across the whole conversation.
 - **user** — per-turn human input.
 - **assistant** — the model's own prior replies (fed back so it "remembers" what it said).
 - **tool** — a tool's result fed back into the conversation. How does the model tell them apart? Roles are tagged (a `role: field`), flattened into delimiter tokens (e.g. `<|system|>`), and the model *learned during training* to trust system over user. That wall is **real but learned, not enforced** — which is exactly why **prompt injection** is possible (cleverly-worded user/tool text can imitate authority). Newer models add hardening (e.g. a non-spoofable operator channel), but the principle stands.
2. **Ownership / specificity** — a system prompt can be **generic** ("you are a helpful assistant") OR **hyper-specific** (a full incident-response procedure). Same role either way. This is where the **3-layer prompt model** lives:
 - ① **BASE / identity prompt** — permanent identity, safety, tool etiquette; whole-agent scope; rarely changes. Usually owned by the host/platform.
 - ② **TASK prompt** — the procedure + output contract for ONE task-type: ordered steps, which-tool-when, required report format. Written ahead of time, injected when that task starts.
 - ③ **CONVERSATION** — the live back-and-forth; changes every turn; built by the loop. Typical composition (where they combine): `system = ① + ②`, `messages = ③`. Keeping ① and the tool list **byte-stable** matters for prompt caching (any change to the prefix invalidates the cache — see [[kb-claude-api]]).

1.9 Context window & statelessness (the hard wall)

- The **context window** (e.g. 200K, 1M tokens) is a **HARD limit** — the API rejects a request that overflows it, or silently drops the oldest content. It counts **everything**: system prompt + full history + tool results + the space reserved for the answer.
- **"Memory" is an illusion maintained by the caller.** Because the model is stateless (§1.1), the agent re-sends the entire message list every call and the model re-reads it from scratch — *amnesia patient + a notebook someone hands them each time*.
- **Overflow is the agent code's problem to solve**, not the model's. The standard strategies:
 - **Truncation** — drop the oldest turns (simple, lossy).
 - **Summarization / compaction** — replace old turns with a model-generated summary (costs an extra call, lossy, but keeps the gist). Many 2026 APIs offer this server-side.
 - **RAG / retrieval** — keep the bulk of knowledge *out* of the prompt in a vector store, and pull in only the relevant chunks per query (§1.2, [[kb-rag-retrieval]]).
- You can't just "use a bigger context" forever: attention is **O(n²)** (§1.2), so doubling context quadruples cost and latency. Bigger windows help, but retrieval/summarization remain necessary at scale. See [[kb-distributed-systems-patterns]] for the general "state lives outside the stateless worker" pattern.

1.10 Agent memory patterns (short-term vs long-term)

"Context window" is *within-conversation*, **short-term** memory. Agents that persist across sessions add explicit **long-term** memory:

- **Scratchpad / working memory** — a file or note the agent writes to *during* a task and re-reads later in the same run (helps long-horizon coherence).
- **Long-term / cross-session memory** — a durable external store (files, a DB, a "memory tool") the agent consults at the start of future sessions. The model decides *what* to write; your code owns the storage backend and access control.
- **Semantic memory via RAG** — past interactions embedded into a vector store and retrieved by similarity, so "memory" scales past what fits in context. The common thread: **short-term memory = the context; long-term memory = an external store/RAG. The durable state is always outside the model.** The model reads it in and writes it out via tools; it never "holds" it.

PART 2 — WORKED EXAMPLE: the WCX SRE Agent

The WCX Engineering Copilot's SRE Agent is a textbook instance of everything above, with one twist that makes it hard to see: **the loop itself lives in a managed service across a REST boundary.** You reason about it from its *evidence in the thread*, not from local source. (See the Trust-status caveat at the top.)

The agent definition (spec.yaml) — where the pieces are declared (§1.4, §1.8)

- The agent is declared in `spec.yaml`: identity, the model, and the tool surface. Two categories of tools appear there, and **the LLM calls both identically** — the difference is only in the *executor*:
 - `tools` — the agent's directly-declared capabilities.
 - `mcpTools` — tools exposed via MCP (§1.4): name→code behind the MCP server, with the signature/docstring/ `Field(description=)` acting as the schema the model sees (signature-as-prompt).
- `spec.yaml` is the ① **BASE prompt / identity layer** owner (§1.8): permanent identity/safety/tool-etiquette, whole-agent scope, rarely changes. Owned by the HOST.

The SRE thread IS the conversation state (§1.6, §1.9)

- There is no local `while` loop and no push stream to "hook into." The agent **WRITES its answer into the SRE thread** (durable storage — see `[[kb-storage-model]]`), not down a wire. The thread is the accumulating transcript (§1.6) — the "notebook" that makes the stateless model appear to remember. **The agent control logic is memoryless; the state lives in the thread.**
- On the WCX side, one turn produces MANY messages in the thread: the user msg + one message per tool call + one per reasoning block + one final answer-text message. **Only the answer-text message grows char-by-char** (it's streamed); tool/reasoning messages appear whole ("discrete"). This is the physical footprint of §1.6's "one turn = many LLM calls" (turn = many messages, not one).

Prompt composition happens IN THE MANAGED SERVICE, not in MCP (§1.8)

- The frontend/backend in this repo send only the agent **NAME**; the instructions are **injected server-side** — prompt composition is not visible in local code.
- The 3 layers (§1.8) map cleanly:
 - ① **BASE** — permanent identity/safety/tool-etiquette; owned by HOST (`spec.yaml`).
 - ② **TASK** — the output of `compose()`: the procedure + output contract for ONE task-type (task-scoped role, ordered steps, which-tools-when, report format). Owned by **MCP**. Example: the `troubleshoot_incident` workflow.
 - ③ **CONVERSATION** — the live thread, changing every turn, built by the host loop.
- **Combining happens at the HOST, never in MCP:** `system = ① + ②`, `messages = ③`. MCP produces ② and stops at `return` — it never assembles the final prompt. The `00-overview` file restates identity inside ② because MCP is **host-agnostic** (it must be self-contained; it can't assume ① is present).
- There is likely ALSO a hidden platform system prompt stacked underneath the visible `system_prompt` — both are "system role" (§1.8); the visible one is the team's scenario-specific persona layer.

Loop timing & the ID-extraction split (Day 3 key insight — §1.5, §1.6, §1.7)

The clearest proof that "reasoning happens inside the loop, one call at a time" (§1.6):

- **Case A (webhook/scheduler kickoff, autonomous):** the incident ID arrives STRUCTURED → the host passes it up-front → no extraction step needed.

- **Case B (human free-text, interactive — "look into incident 12345"):** the LLM extracts the ID *inside iteration 1 of the loop*. There is NO pre-loop phase — the LLM only ever acts *inside* the loop. It then invokes `troubleshoot_incident(12345) ; compose()` produces ② **mid-loop**; ② is appended; iteration 2 sees the full workflow.
- **Both cases converge** to the same state (an LLM holding ①+② following the workflow); only the **TIMING of when ② exists** differs. Whether `troubleshoot_incident` is framed as a prompt-injector or a tool is a **host-design choice**. This is §1.7's autonomous-vs-interactive axis and §1.5's loop, made concrete.

Autonomous vs interactive on the WCX side (§1.7)

- **Portal chat = interactive** — a human steers turn-by-turn (Case B kickoff).
- **SRE Agent = autonomous** — fires on a **cron/scheduler or an incident trigger** (webhook) and runs to completion unattended (Case A kickoff).
- Same agent definition; only the kickoff and when-it-stops-for-input differ.

The stop/act division on the WCX side (ties to §1.5, §1.6)

- **LLM = stateless oracle that DECIDES** via `stop_reason` — `tool_use` (run + loop) vs `end_turn` (stop). The multi-tool investigations you see in a WCX incident are the SAME `tool_use` signal firing repeatedly (§1.5), not a special "keep going" state. (Corrected: `text≠end_turn` — narration before a tool call still carries `stop_reason == tool_use`.)
- **The managed-service loop = the stateful orchestrator** that runs the tool, appends the result to the thread, and re-sends. The control logic is near-memoryless; the state is the thread (§1.6). Reason+call+call here is a **ping-pong across the REST boundary, not inside the LLM**.
- The **consequence** of a decision (a tool call + its result) is written to the thread and streamed to the browser as a **tool card** — visible. Pure internal deliberation may be invisible. (How that stream reaches the browser: `[[kb-streaming-pipeline]]`.)
- Context/overflow on the WCX side is the AGENT LOOP's job (§1.9): it decides how much thread history to re-send each call (the old Foundry path used `truncation="auto"`).

PART 3 — transferable takeaways (the interview/next-job version)

1. **An LLM is a stateless next-token predictor.** It can't *do* or *remember* anything; every agent capability comes from the code around it. "Agent = LLM decides, agent code does."
2. **Tool use is a handshake, not execution.** The model emits a `tool_use` block and STOPS; your harness runs the code and feeds back a `tool_result`. Vague tool descriptions cause wrong tool calls — the schema is prompt (signature-as-prompt in MCP).
3. **The loop stops on OUTPUT TYPE, not content.** `stop_reason == tool_use` → run + loop; `end_turn` → stop. Text appearing ≠ done — a response can carry text *and* `tool_use` together (narration before a call).
4. **One turn = many LLM calls.** Reason→tool→reason is a ping-pong across the API boundary; each call is fresh and re-reads the entire growing transcript. The model holds nothing between calls.
5. **"Memory" is re-sending the transcript.** The model is stateless; the caller maintains context. The window is a hard, $O(n^2)$ wall — manage overflow with truncation, summarization, or RAG. Short-term = context; long-term = external store.
6. **Prompt "system" is a role (authority), not a level.** The 3-layer model — identity ① / task ② / conversation ③ — is composed at the host: `system = ① + ②`, `messages = ③`. The role wall is learned, not enforced (hence prompt injection).
7. **Know the landscape by shape, not brand.** ReAct is the pattern; MCP is the 2026 cross-vendor tool standard; LangChain/LlamaIndex/AutoGen/vendor SDKs are pre-built loops. Classify any system by single-vs-multi-agent and autonomous-vs-interactive.
8. **When the loop lives behind a service boundary (like WCX's SRE Agent), you reason about it from its evidence** — the messages it writes to the thread — not from local source. The mechanics are still the standard `stop_reason` contract; the agent is memoryless, the thread holds the state.